



SEC2213
Defense & Defiance:
Building and Breaking Asymmetric Cryptosystems

Track4
Mini Crypto Wallet

6 Apr 2026

Student Name	Id Number
Layan Saleem	445000832
Shoog Albakri	445003096
Samar Siraj	44610081
Ruba Albajaly	445000256
Saja Azhar	44610340

Table of Contents

Introduction	3
1. System Implementation	3
1.1 Private Key Generation.	3
1.1.2 Security Purpose	4
1.2 Public Key and Wallet Address Generation	4
1.2.2 Security Purpose	5
1.3 GUI, Deposit, and Withdrawal	5
2. Brute Force Attack Simulation	6
2.1 Methodology:	6
2.2 Experimental Outcome:	6
2.3 Security Insight:	6
3. Complexity & Performance Analysis	6
4. Ethical Analysis: Government Backdoors	7
4.1 Systemic Risk:	7
4.2 Critical Finding:	7
4.3 Conclusion:	7
5. AI-Based Analysis:	8
5.1 Data Collection	8
5.2 Model Training	8
5.3 Results & Evaluation	8
5.4 Visualization	9
5.5 Security Insight	9
6. Bonus Challenge	10
6.2 Timing Attack on RSA Algorithm	10
6.3 Output	11
6.4 Graph Analysis	11
Conclusion	12
The distribution of the work	13

Introduction

To protect digital assets and secure user information, Cryptocurrency Wallets strongly rely on utilizing cryptography to generate random and unpredictable Private Keys (keys). Any vulnerability found in the random generation process could lead to serious security issues where hackers will take advantage to compromise Wallets.

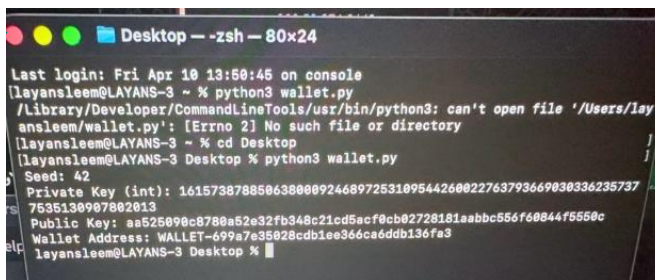
In addition, this project highlights how weak or predictable seed sources used to create Private Keys can lead to Wallet Security being compromised despite using strong encryption standards such as 256-bit keys. The project simulates a complete process of generating a Wallet's Private Key, deriving the associated Public Key and Wallet Address from the Private Key, and testing an entire system against Brute Force based attacks. This project will also explore analytical techniques using Artificial Intelligence (AI) to identify statistical patterns created from weakly generated random seeds and evaluates the performance of using these model types to predict cryptographic values.

Timestamps will also be simulated against the RSA algorithm in this project to illustrate how execution time can be used as a potential leakage point for revealing sensitive cryptographic material. Finally, the proposal includes a conversation about the ethical concerns of introducing backdoors or vulnerabilities to encryption systems from supporting government agencies.

To see all Code.py go to <https://github.com/layansalym-creator/Track4-.git>

1. System Implementation

1.1 Private Key Generation



```
Desktop - zsh - 80x24
Last login: Fri Apr 10 13:50:45 on console
layansleem@LAYANS-3 ~ % python3 wallet.py
/Library/Developer/CommandLineTools/usr/bin/python3: can't open file '/Users/layansleem/wallet.py': [Errno 2] No such file or directory
layansleem@LAYANS-3 ~ % cd Desktop
layansleem@LAYANS-3 Desktop % python3 wallet.py
Seed: 42
Private Key (int): 161573878850638009246897253109544260022763793669030336235737
7635130907802013
Public Key: aa525090c8780a52e32fb348c21cd5acf0cb2728181aabb056f60844f5550c
Wallet Address: WALLET-699a7e35028c0b1ee366ca0ddb136fa3
layansleem@LAYANS-3 Desktop %
```

1. This code simulates the generation of a cryptocurrency wallet's Private Key using Python's random library.

The program demonstrates how weak or predictable seeds can create security vulnerabilities in cryptographic systems.

- First, the random library is imported to generate pseudo-random numbers.

- A function called `generate_wallet(seed)` is then created to generate the wallet key.
- Inside the function, `random.seed(seed)` initializes the random number generator using a fixed seed value.
- Then, `random.getrandbits(256)` generates a 256-bit Private Key.

The program uses the seed value 42 as an example and prints the generated Private Key in both integer and hexadecimal formats.

1.1.2 Security Purpose

The code demonstrates a Weak Seed Vulnerability.

Since the same seed always produces the same Private Key, an attacker who can predict the seed may regenerate the key and compromise the wallet's security.

- Key Generation:

The core of the system relies on generating a secure 256-bit private key using the `random.getrandbits(256)` function, which ensures the creation of a mathematically strong and complex key.

- Hashing Process:

To derive a public address, the system utilizes the SHA-256 cryptographic hash algorithm through Python's `hashlib` library, ensuring the highest level of data integrity.

1.2 Public Key and Wallet Address Generation

```
PS C:\Users\Study\OneDrive\Desktop\wallet> python wallet.py
Seed: 42
Private Key (int): 16157387885063800092468972531095442600227637936690303362357377535130907802013
Public Key: aa525090c8780a52e32fb348c21cd5acf0cb02728181aabb556f60844f5550c
Wallet Address: WALLET-699a7e35028cdb1ee366ca6ddb136fa3
PS C:\Users\Study\OneDrive\Desktop\wallet>
```

1. This part of the project focuses on generating a simulated Public Key and Wallet Address from the generated Private Key.
 - First, the Private Key is converted into a string format and hashed using the SHA-256 algorithm through Python's `hashlib` library.
 - The resulting hash value represents the simulated Public Key.
 - After that, the Public Key is hashed again using SHA-256 to generate a Wallet Address.
 - The output is shortened and combined with the prefix "WALLET-" to simulate a cryptocurrency wallet address format.

This process demonstrates the relationship between the Private Key, Public Key, and Wallet Address in a simplified cryptocurrency wallet system.

1.2.2 Security Purpose

The hashing process helps simulate how wallet systems protect sensitive information and generate unique wallet identifiers.

Using SHA-256 ensures that the generated values are consistent and difficult to reverse, which demonstrates an important concept in cryptographic systems.

1.3 GUI, Deposit, and Withdrawal

An interactive GUI was developed using Streamlit to make the cryptocurrency wallet simulation easier to use and more realistic. The interface displays important wallet information such as the Wallet Address, Public Key, and the current wallet balance.

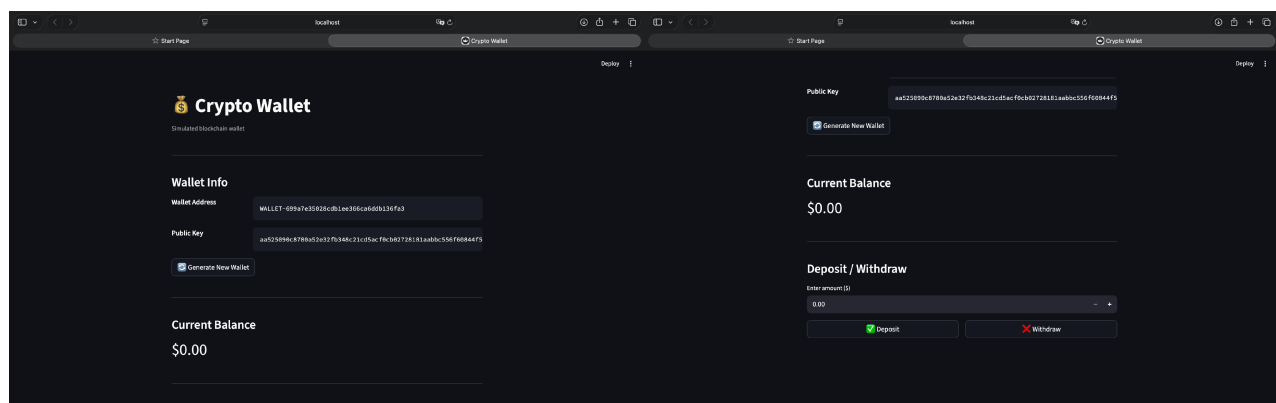
The application includes an input field where the user can manually enter an amount to deposit or withdraw. When the user clicks the Deposit button,

the entered amount is added to the current balance and the updated balance is displayed immediately. When the Withdraw button is pressed,

the amount is deducted from the balance only if sufficient funds are available.

The system also includes a simple validation mechanism to prevent withdrawing more than the available balance. In this case, an error message is shown to the user, while successful transactions display confirmation messages.

The purpose of this feature is to transform the project from a simple command-line simulation into a more interactive application that better represents how real cryptocurrency wallets manage balances and transactions.



2. Brute Force Attack Simulation

2.1 Methodology:

The attack script is designed to recover the private key by systematically iterating through various "seeds" and generating corresponding addresses until a match with the target address is found.

2.2 Experimental Outcome:

During the simulation, the script successfully compromised the system and recovered the private key using a Weak Seed (42).

2.3 Security Insight:

The experiment highlights that even with a 256-bit encryption standard, the system remains vulnerable if the underlying seed is predictable, allowing for a near- instantaneous recovery of the key.

3. Complexity & Performance Analysis

The following data tracks the efficiency of the brute force attack across different key strengths, demonstrating how a predictable seed affects security regardless of the bit length:

KEY STRENGTH (BITS)	FOUND SEED	TIME TAKEN (SECONDS)	STATUS
8-BIT	38	0.000817 s	Success
16-BIT	42	0.000933 s	Success
32-BIT	42	0.000849 s	Success
256-BIT	42	0.0006 s	Success

5. AI-Based Analysis:

To do additional analysis of the system's weaknesses, we created a Linear Regression model based on AI algorithms.

5.1 Data Collection

We created a dataset by operating the wallet system for a number of different seed values. We used sequentially ordered seed values, ranging from 0-1000. For each seed value, the associated private key that was generated was stored in a CSV file (data.csv). The generated data is the training data for the AI.

5.2 Model Training

The AI model will learn the relationship between seed values and corresponding private keys.

The AI model was developed with the following components:

- Inputs (X): Seed values
- Outputs (Y): Generated private keys

5.3 Results & Evaluation

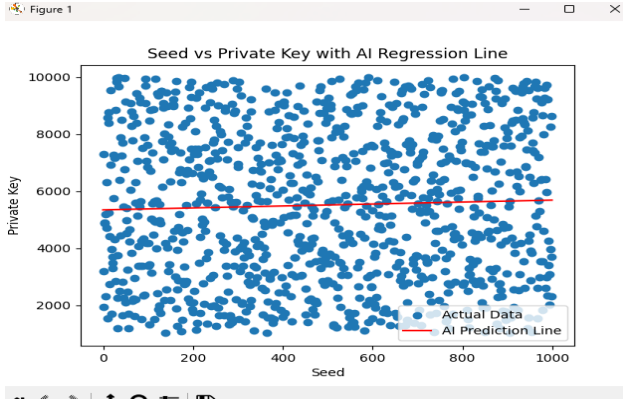
- The Model was tested with a new seed value and predicted the private key that would have been generated had the model been used.

```
PS C:\Users\Smart\OneDrive\Desktop\crypto_project> python ai_model.py
Predicted Private Key: [5360.09723457]
Real Private Key: 2824
```

Test results were then compared between reported and actual key values.

- The model was of limited value for predicting the exact private key created with that seed value. Significant deviations between predicted and actual key values were common.
- The outcomes described above are expected due to the following factors:
 - Key generation uses pseudo-random functions.
 - The relationship between the seed value and the key created is not linear.
 - Linear Regression will not be a suitable function to model complex random behavior.

5.4 Visualization



The relationship between private keys and seed values was examined through the use of a scatter plot, and the regression line created by the AI model was plotted to show the prediction trend.

Based on the graph, there were:

- Several scattered data point.
- No discernible linear relationship.
- A weak trend line that does not accurately reflect the underlying data.

5.5 Security Insight

The AI model failed in determining exact private keys; however, the experiment offered insight into the following areas:

- There are patterns in the system originated from weak seeds.
- There is partial detection of trends by the AI model even though the actual predictions were not accurate.
- The combination of trends generated by weak seeds with brute-force attacks will reduce the time of performing an attack.

The findings of the AI experiment indicate that machine learning on its own does not have the capability to break arbitrarily chosen cryptographic keys using pseudo-random functions, yet the results of this experiment indicate that weak randomness can produce exploitable vulnerabilities within any given system.

For this reason, secure generation of random seed values and unpredictability is required to assure system integrity.

6. Bonus Challenge

6.1 Timing Attack on RSA Algorithm

```
1 from wallet import private_key
2 import time
3 import numpy as np
4 from sklearn.linear_model import LinearRegression
5
6
7 def rsa_decrypt_vulnerable(ciphertext, private_key, mod):
8     result = 1
9     for bit in bin(private_key)[2:]:
10         result = (result * result) % mod
11         time.sleep(0.0001)
12
13         if bit == '1':
14             result = (result * ciphertext) % mod
15             time.sleep(0.0002)
16
17     return result
18
19
20
21 print("[*] Collecting timing data for training...")
22
23 X_times = []
24 y_lengths = []
25
26 for length in [8, 16, 24, 32]:
27     for _ in range(10):
28         key = np.random.randint(1 << (length - 1), (1 << length) - 1)
29
30         start = time.perf_counter()
31         rsa_decrypt_vulnerable(12345, key, 1000000007)
32         elapsed = time.perf_counter() - start
33
34         X_times.append([elapsed])
35         y_lengths.append(length)
36
37 model = LinearRegression()
38 model.fit(X_times, y_lengths)
39
40 print("[+] Model trained successfully.")
41
42
43
44 print("\n--- Simulating attack on unknown key ---")
45
46 secret_length = 20
47 secret_key = np.random.randint(
48     1 << (secret_length - 1),
49     (1 << secret_length) - 1
50 )
51
52 with open("key.txt", "w") as f:
53     f.write(str(secret_key))
54
55 start = time.perf_counter()
56 rsa_decrypt_vulnerable(12345, secret_key, 1000000007)
57 measured_time = time.perf_counter() - start
58
59 predicted = model.predict([[measured_time]])[0]
60
61 print(f"Measured time: {measured_time:.6f} seconds")
62 print(f"Actual key length: {secret_length} bits")
63 print(f"Predicted by AI: {round(predicted)} bits")
```

This code demonstrates a Timing Attack on the RSA algorithm by measuring the decryption execution time for different key lengths.

The collected timing data is used to train a Linear Regression AI model that predicts the private key length based on the measured execution time.

6.2 Timing Attack on RSA Algorithm

The code generates RSA keys with different lengths, measures the decryption execution time, and stores the timing data.

Then, it trains a Linear Regression model to learn the relationship between execution time and key length.

Finally, the model predicts the length of an unknown secret key based on the measured decryption time.

6.3 Output

```
Desktop --zsh-- 80x23

Last login: Tue May  5 12:24:55 on ttys000
[layansleem@LAYANS-3 ~ % cd Desktop
[layansleem@LAYANS-3 Desktop % python3 wallet.py && python3 bonus.py
Seed: 42
Private Key (int): 1615738788506380009246897253109544260022763793669030336235737
7535130907802013
Public Key: aa525090c8780a52e32fb348c21cd5acf0cb02728181aabb556f60844f5550c
Wallet Address: WALLET-699a7e35028cdb1ee366ca6ddb136fa3
[*] Collecting timing data for training...
[+] Model trained successfully.

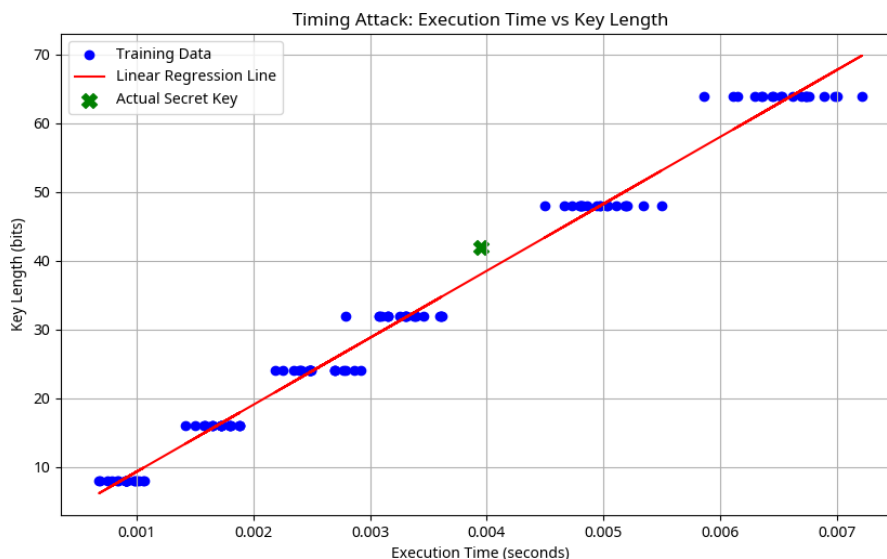
--- Simulating attack on unknown key ---
Measured time: 0.004877 seconds
Actual key length: 20 bits
Predicted by AI: 19 bits
layansleem@LAYANS-3 Desktop %
```

The figure shows the execution result of the Timing Attack simulation on the RSA algorithm. The program successfully generated RSA key information, collected timing data, and trained the AI model.

During the attack simulation, the actual secret key length was 20 bits, while the AI model predicted it as 19 bits based on the measured execution time.

This result demonstrates that timing information can be used to estimate sensitive cryptographic details with high accuracy.

6.4 Graph Analysis



The graph shows the relationship between RSA decryption time and private key length during the Timing Attack experiment.

- The blue points represent the training data.
- The red line represents the Linear Regression model.
- The green marker shows the actual secret key.

The results show that execution time can help predict the key length.

Conclusion

This project illustrated why effective randomness and how it's implemented are critical to the security of cryptographic systems. Analysis of the experimental results demonstrated that, while strong encryption algorithms such as 256-bit key generation and RSA provide a great deal of mathematical certainty, various factors such as weak seeds and side-channel information can reduce the protection provided by these algorithms.

The experiment with cryptocurrency wallet simulation clearly demonstrated how predictable seed values can allow an attacker to recover private keys through brute-force methods in a short period. Additionally, the AI-based analysis showed how weak random numbers can be analysed by using machine learning techniques that may be able to detect certain patterns; however, they do not have the ability to accurately detect secure cryptographic keys.

The Timing-Attack Experiment against RSA also indicated other ways that information can be leaked through execution time analysis. By monitoring timing behaviour of an RSA execution, attackers may accurately estimate many types of sensitive cryptographic information.

Overall, the results of this project strongly describe that the security of cryptographic systems is dependent upon more than just the strength of the encryption algorithms; the generation of secure random numbers, the quality of the implementation and the fact that there are no intentional weaknesses are also essential to the security of the systems. Therefore, reliable random numbers, and strong end-to-end encryption are necessary for protecting the privacy of the users and for ensuring the digital world is secure.

The distribution of the work

<i>Member</i>	<i>Task(s)</i>
<i>Layan Saleem</i>	- Implementation - Report Review - GUI, Deposit, and Withdrawal
<i>Shoog Albakri</i>	- Implementation - Report Review
<i>Samar Siraj</i>	- Attack - The Video - Report Review
<i>Ruba Albajaly</i>	- AI-Based Analysis - Report Preparation - Presentation Preparation
<i>Saja azhar</i>	- Bonus Challenge - Report Review